DSAT by MPS

About Algorithms

An algorithm is a finite sequence of well-defined instructions used to solve a computational problem. It's designed to take input and produce output by transforming the input step-by-step. Algorithms are fundamental to computer science and are used to ensure solutions are not only correct but also efficient. Their efficiency is often measured in terms of time and space complexity, using notations like Big O (e.g., O(n log n)) to describe performance as input size grows.

Insertion Sort

Insertion Sort builds the final sorted array one item at a time by comparing each new element to those already sorted and inserting it in the correct position. It is efficient for small or nearly sorted datasets with worst-case time complexity $O(n^2)$.

Problem:
Sort the array A = [5, 2, 4, 6, 1, 3] using Insertion Sort.

Solution (Step-by-step):

1. i=1, key=2 → Compare with 5 → shift 5 → insert 2 → [2,5,4,6,1,3]
2. i=2, key=4 → Compare with 5 → shift 5 → insert 4 → [2,4,5,6,1,3]
3. i=3, key=6 → No shift needed → [2,4,5,6,1,3]
4. i=4, key=1 → shift 6,5,4,2 → insert 1 → [1,2,4,5,6,3]
5. i=5, key=3 → shift 6,5,4 → insert 3 → [1,2,3,4,5,6]

Final Output: [1, 2, 3, 4, 5, 6]

Merge Sort

Merge Sort is a divide-and-conquer algorithm that recursively divides the array into halves, sorts each half, and merges them back in sorted order. It has a consistent time complexity of O(n log n) and is efficient for large datasets.

Problem:
Sort the array A = [5, 2, 4, 6, 1, 3] using Merge Sort.

Solution (Step-by-step):

1. Divide → [5,2,4] and [6,1,3]
2. Divide → [5] [2,4] and [6] [1,3]
3. Divide → [2] [4], [1] [3]
4. Merge → [2,4], then [5,2,4] → [2,4,5]

5. Merge → [1,3], then [6,1,3] → [1,3,6]
6. Final merge → [2,4,5] and [1,3,6] → [1,2,3,4,5,6]

Final Output: [1, 2, 3, 4, 5, 6]

Recurrence Relation

A recurrence relation defines a function in terms of its value on smaller inputs, often used to express the time complexity of recursive algorithms. Solving it gives the overall runtime; common techniques include substitution, recursion tree, and master method.

Problem:
Solve the recurrence: $T(n) = 2T(n/2) + n$

Solution:
Using the Master Theorem:

- $a = 2$, $b = 2$, $f(n) = n$
- Since $f(n) = \Theta(n^{\log_2 2}) = \Theta(n)$, it fits Case 2 of Master Theorem.
  ✅ Therefore, $T(n) = \Theta(n \log n)$

Hashing – Double Hashing

Hashing is a technique to map keys to indices in a hash table for efficient data retrieval. Double hashing resolves collisions using two hash functions: the second one determines the step size for probing.

Problem:
Insert keys [10, 22, 31, 4, 15, 28, 17, 88, 59] into a table of size 11 using:

- $h_1(k) = k \% 11$
- $h_2(k) = 1 + (k \% 10)$
- $h(k, i) = (h_1(k) + i \cdot h_2(k)) \% 11$

Solution:

- 10 → h(10,0)=10
- 22 → h(22,0)=0
- 31 → h(31,0)=9
- 4 → h(4,0)=4
- 15 → h(15,0)=4 (occupied), h(15,1)=9 (occupied), h(15,2)=2
- 28 → h(28,0)=6
- 17 → h(17,0)=6 (occupied), h(17,1)=0 (occupied), h(17,2)=7
- 88 → h(88,0)=0 (occupied), ... → h(88,4)=8
- 59 → h(59,0)=4 (occupied), ... → h(59,4)=1

Final Table:
[22, 59, 15, –, 4, –, 28, 17, 88, 31, 10]

Binary Search Tree (BST)

A Binary Search Tree is a binary tree where each node has a key, and all keys in the left subtree are smaller, while all keys in the right subtree are larger. It allows efficient operations like search, insert, and delete in O(log n) average time.

Problem:
Construct a BST from the sequence: [8, 3, 10, 1, 6, 14, 4, 7, 13]

Solution (Insertion order):

- Insert 8 → root
- Insert 3 → left of 8
- Insert 10 → right of 8
- Insert 1 → left of 3
- Insert 6 → right of 3
- Insert 14 → right of 10
- Insert 4 → left of 6
- Insert 7 → right of 6
- Insert 13 → left of 14

Final BST:

```markdown
    8
   / \
  3   10
 / \   \
1   6   14
   / \  /
  4  7 13
```

Binary Search Tree Deletion

BST deletion has three cases:

1. Leaf node – just remove it.
2. One child – replace the node with its child.
3. Two children – replace the node with its inorder successor (smallest in right subtree) or inorder predecessor.

Problem:
Delete node 10 from the BST built from [8, 3, 10, 1, 6, 14, 4, 7, 13]

Solution:

- Node 10 has one right child (14)
- Replace node 10 with 14
- Now insert 13 remains as left child of 14

Updated BST:

```markdown
    8
   / \
  3   14
 /\  /
1 6 13
  /\
 4 7
```

Binary Search Tree – Insertion Problems

1. Insert the keys [20, 10, 30, 5, 15, 25, 35] into a BST.
2. Insert the keys [50, 40, 60, 35, 45, 55, 70] into a BST and draw the final structure.

Binary Search Tree – Deletion Problems

1. Delete the key 10 from the BST constructed with [20, 10, 30, 5, 15].
2. Delete the key 60 from the BST built using [50, 40, 60, 55, 70].

DSAT-80%

🔴⚫ Red-Black Tree Properties Recap

1. Every node is either red or black.
2. The root is black.
3. All leaves (NIL nodes) are black.
4. Red nodes cannot have red children (no two reds in a row).
5. Every path from a node to its descendant NIL nodes contains the same number of black nodes.

✅ Initial Tree

Let's start with an empty tree and insert nodes in this order: 10, 20, 30

🌱 Insertion 1: Insert 10

- 10 becomes the root.
- Root is always black.

```
css

10(B)
```

🌿 Insertion 2: Insert 20

- 20 is red and goes to the right of 10.
- No violations occur (parent is black).

```
scss

10(B)
  \
  20(R)
```

🧨 Insertion 3: Insert 30 (causes violation)

- Insert 30 as red to the right of 20.

- Now, we have two consecutive red nodes (20 and 30) — violation of property 4.
- Fix: Perform left rotation at 10 and recolor.

Before fix:

```
scss

  10(B)
    \
    20(R)
      \
      30(R)
```

After left-rotation at 10 and recoloring:

```
css

  20(B)
 /   \
10(R)  30(R)
```

✅ Now it satisfies all red-black properties.

❌ Deletion 1: Delete 10

- 10 is a red leaf → direct removal is safe, no fix required.

After deletion:

```
scss

  20(B)
    \
    30(R)
```

❌ Deletion 2: Delete 20

- 20 is black and has one red child (30).
- Replace 20 with 30 and color 30 black to maintain black-height.

Final tree:

```
css

  30(B)
```

✅ Summary

- Insertions: 10, 20, 30 (required one rotation and recolor).
- Deletions: 10 (no fix), 20 (color adjustment only).

- A 10-node initial Red-Black Tree
- 2 insertions
- 2 deletions

🧩 Problem Statement:

Given the following Red-Black Tree containing 10 keys:

```
css

          20(B)
        /      \
     10(R)     30(B)
    /   \        \
 5(B) 15(B)     40(R)
           /     \
        35(B)   50(B)
```

Tree Keys: 5, 10, 15, 20, 30, 35, 40, 50 (that's 8 keys visible, 2 NIL leaves implied for RBT structure)

📝 Perform the following operations:

1. Insert 12
2. Insert 45
3. Delete 30
4. Delete 10

🎯 Objective:

After each operation, maintain Red-Black Tree properties:

- Balance using rotations if needed
- Recolor nodes accordingly

DSAT-80%

📌 Gist: Augmenting Data Structures

Augmenting a data structure means enhancing a basic structure (like a Binary Search Tree, BST) by adding extra information to each node to support more advanced operations efficiently.

🔍 Why augment?
To enable fast queries like:

- Order statistics (e.g., kth smallest element)
- Interval overlapping
- Range queries
- Rank of a value

🧠 How?

- Store auxiliary info like subtree size, min/max values, or overlapping intervals in each node.
- Update these fields during insertion/deletion/rotation, so they always stay correct.

📚 Most commonly used with:

- Red-Black Trees
- AVL Trees
- Segment Trees

🧩 Sample Problem:

You are given an augmented Red-Black Tree where each node contains:

- `key` (integer)
- `size` = number of nodes in the subtree rooted at that node

Problem:
Design a function `SELECT(root, k)` that returns the kth smallest key in the tree in O(log n) time.

📌 Gist: Disjoint-Set (Union-Find) Data Structure

A Disjoint-Set is a data structure that keeps track of a partition of a set into disjoint (non-overlapping) subsets. It supports efficient operations to:

1. MAKE-SET(x) – Create a new set containing only x.
2. FIND-SET(x) – Return the "representative" (leader) of the set containing x.
3. UNION(x, y) – Merge the sets containing x and y.

💡 Key Applications:

- Finding connected components in a graph

- Kruskal's algorithm for Minimum Spanning Trees
- Cycle detection in undirected graphs
- Dynamic connectivity problems

⚡ Optimizations:

- Union by rank: Always attach the smaller tree to the root of the larger tree.
- Path compression: Flatten the structure during `FIND-SET` to make future queries faster.

With both optimizations, operations are very close to O(1) (amortized) — specifically O(α(n)), where α(n) is the inverse Ackermann function.

🧩 Sample Problem:

You are given a list of friendships between people:

```python
friendships = [(1, 2), (3, 4), (2, 3), (5, 6)]
```

Using Disjoint-Set, determine how many friend circles (connected groups of friends) exist.

👀 *Hint*: Use `MAKE-SET` for each person, and `UNION` for each friendship. Then count how many unique sets remain using `FIND-SET`.

🧠 Kruskal-Based Application Using Disjoint-Set (Algorithm Only)

Let's use the friend circle problem as an application of Kruskal's algorithm principles with a Disjoint-Set (Union-Find) data structure.

💬 Problem Restated:

Given a list of friend pairs, determine how many friend circles (connected components) exist.

Input:

```ini
friendships = [(1, 2), (3, 4), (2, 3), (5, 6)]
```

⚙️ Algorithm (Inspired by Kruskal's approach):

1. Initialization
   For each person `x` appearing in the friendships, do:
   - `MAKE-SET(x)`

2. Process Friendships (Like Kruskal processes edges)
   For each pair `(u, v)` in the list:
   - If `FIND-SET(u) ≠ FIND-SET(v)`, then:
     - `UNION(u, v)`
3. Count Friend Circles
   After all unions:
   - For each person `x`, compute `FIND-SET(x)` and store the unique representatives.
   - The number of unique representatives is the number of friend circles.

🧪 Walkthrough with the Example:

friendships = [(1, 2), (3, 4), (2, 3), (5, 6)]

- MAKE-SET for all: `{1}, {2}, {3}, {4}, {5}, {6}`
- UNION(1,2) → {1,2}
- UNION(3,4) → {3,4}
- UNION(2,3) → merges {1,2} and {3,4} → {1,2,3,4}
- UNION(5,6) → {5,6}

✅ Final sets: `{1,2,3,4}`, `{5,6}` → 2 friend circles

This mimics Kruskal's idea: process all relationships (edges) in increasing order (or any order, since no weights here), and merge sets — just like building a minimum spanning forest.

DSAT-80%

🌳 Disjoint-Set as Trees (Rooted Tree Representation)

Instead of using linked lists, Disjoint-Set can be implemented using rooted trees, where:

- Each node points to its parent
- The root of the tree is the representative of the set

🛠️ Structure & Operations

1. MAKE-SET(x)

- Create a new tree with a single node `x`.
- `parent[x] = x`
- Optional: `rank[x] = 0` (for optimization)

2. FIND-SET(x)

- Traverse up the parent pointers to find the root.
- With path compression: make all nodes on the path point directly to the root (flattens the tree).

3. UNION(x, y)

- Link the roots of `x` and `y`.
- With union by rank: always attach the smaller tree under the larger one (based on `rank`).

🔍 Example

Let's say we have sets `{1}, {2}, {3}, {4}`
We perform:

1. `MAKE-SET(1), MAKE-SET(2), ... MAKE-SET(4)`
   → Trees:

```
1   2   3   4
```

2. `UNION(1,2)` → make 1 the parent of 2

```
1
|
2
```

3. `UNION(3,4)` → make 3 the parent of 4

```
3
|
4
```

4. `UNION(1,3)` → attach one root under the other (say 1 under 3)

```arduino
  3
 /
1
|
2
|
4 (if 4 was compressed via find)
```

⚡ Optimizations

- Path Compression (in FIND-SET)
  Flattens trees → next queries are faster.
- Union by Rank (in UNION)
  Keeps tree short → prevents tall, linear chains.

Together, these lead to nearly constant time per operation: O(α(n)), where α is the inverse Ackermann function (very slow-growing).

🌐 1. Breadth-First Search (BFS)

🔍 Purpose:

- Explore all vertices reachable from a source vertex
- Finds shortest path (in terms of edges) in unweighted graphs

---

⚙️ Algorithm (for graph G = (V, E))

```text
BFS(G, s):
  for each vertex v in V:
    color[v] = WHITE
    distance[v] = ∞
    parent[v] = NIL

  color[s] = GRAY
```

```
distance[s] = 0
parent[s] = NIL

Q = empty queue
ENQUEUE(Q, s)

while Q is not empty:
  u = DEQUEUE(Q)
  for each neighbor v of u:
    if color[v] == WHITE:
      color[v] = GRAY
      distance[v] = distance[u] + 1
      parent[v] = u
      ENQUEUE(Q, v)
  color[u] = BLACK
```

🧪 Example:

Graph:

```
css

  A
 / \
B   C
 \ /
  D
```

Start BFS from `A`:

1. Visit A → B, C
2. Visit B → D
3. Visit C → D (already seen)
4. Visit D → done

Shortest path from A to D = A → B → D (or A → C → D), with 2 edges

---

🚀 2. Dijkstra's Algorithm

🔍 Purpose:

- Find the shortest path (by weight) from a source to all other vertices in a weighted, non-negative graph

---

⚙️ Algorithm (Min-Heap version)

```
text

DIJKSTRA(G, s):
  for each vertex v in V:
    distance[v] = ∞
```

```
  parent[v] = NIL

distance[s] = 0

Q = min-priority queue with all vertices

while Q is not empty:
  u = EXTRACT-MIN(Q)
  for each neighbor v of u:
    if distance[v] > distance[u] + weight(u, v):
      distance[v] = distance[u] + weight(u, v)
      parent[v] = u
      DECREASE-KEY(Q, v)
```

🧪 Example:

Graph:

```
css

   A
  / \
 B   C
(1) (4)
  \ /
   D
   (2)
```

Weights:

- A-B = 1
- A-C = 4
- B-D = 2
- C-D = 1

From A:

- A → B → D = 1 + 2 = 3
- A → C → D = 4 + 1 = 5
- So shortest path to D is through B: A → B → D

📊 Time Complexity:

- BFS: O(V + E)
- Dijkstra: O((V + E) log V) using a min-heap (e.g., priority queue)

🟦 1. Breadth-First Search (BFS)

Graph (Unweighted):

```
css

   A
  / \
 B   C
  \ /
   D
```

Start from: `A`

- Level 0: `A` (source)
- Level 1: `B`, `C` (neighbors of A)
- Level 2: `D` (neighbor of B/C)

⏱️ Step-by-step Traversal:

```
mathematica

Queue: [A]
Visit A   Queue: [B, C]   distance[B]=1, distance[C]=1
Visit B   Queue: [C, D]   distance[D]=2
Visit C   (D already discovered)
Visit D   Done

Parent Map:
B   A
C   A
D   B or C
```

🔚 Shortest path from A to D:

- A → B → D (or A → C → D)

🟨 2. Dijkstra's Algorithm

Graph (Weighted):

```
css

    A
   / \
 (1)  (4)
  B    C
   \  /
  (2)(1)
    D
```

Edge Weights:

- A → B = 1
- A → C = 4
- B → D = 2
- C → D = 1

🎯 Start from `A`:

Step	Current Node	Update Neighbors	Distance Array (A, B, C, D)
1	A	B=1, C=4	(0, 1, 4, ∞)
2	B	D=1+2=3	(0, 1, 4, 3)
3	D	C=MIN(4, 3+1)=4	(0, 1, 4, 3)
4	C	—	(0, 1, 4, 3)

🔚 Shortest Paths:

- A → B → D = 3
- A → C = 4
- A → C → D = 5 (not optimal)

DSAT-80%

🔹 Problem 1: BFS – Shortest Path in Unweighted Graph

Given:
Graph as adjacency list:

```text
A: B, C
B: A, D
C: A, D
D: B, C, E
E: D
```

Task:
Using BFS, find the shortest path from A to E, and output the path and number of edges.

🔹 Problem 2: Dijkstra – Shortest Weighted Path

Given:
A weighted graph with the following edges:

```css
A  B (1), A  C (5)
B  C (2), B  D (4)
C  D (1)
```

Task:
Using Dijkstra's Algorithm, compute the shortest distances from `A` to all other nodes. Also, output the shortest path to node `D`.

🔹 Problem 3: Disjoint-Set – Count Connected Components

Given:
Friendships (undirected connections):

```scss
(1, 2), (2, 3), (4, 5), (6, 7), (7, 8), (9, 10)
```

Task:
Using Disjoint-Set (Union-Find) with path compression, count how many friend groups (connected components) exist among people numbered 1 through 10.

🔹 Problem 4: Disjoint-Set – Cycle Detection in Undirected Graph

Given:
Graph edges:

```scss
(1, 2), (2, 3), (3, 4), (4, 2)
```

Task:
Use Disjoint Set with Union-Find to detect whether this graph contains a cycle.

💡 *Hint*: For each edge, if the vertices are already in the same set, adding the edge would form a cycle.

🚰 1. Ford-Fulkerson Method

🔍 Goal:

Find the maximum flow from a source `s` to a sink `t` in a flow network.

🌊 Key Concepts:

- Capacity: Max units allowed on each edge
- Flow: Actual units sent on an edge
- Residual Graph: Tracks remaining capacity
- Augmenting Path: A path from `s` to `t` in the residual graph with available capacity

---

⚙️ Algorithm Steps:

1. Initialize all flows to 0
2. While there exists an augmenting path from `s` to `t` in the residual graph:
   - Find the minimum residual capacity along that path (called bottleneck)
   - Augment the flow along the path by this bottleneck value
   - Update the residual graph (add backward edges for cancellation)
3. Repeat until no augmenting paths exist

✅ At the end, the total flow pushed from `s` to `t` is maximum flow

---

🧪 Example:

Graph:

```
css

s   A (10)
s   B (5)
A   B (15)
A   t (10)
B   t (10)
```

Run Ford-Fulkerson (paths, updates) → Max Flow = 15

🧩 2. Bipartite Maximum Matching

🔍 Goal:

Find the maximum number of matchings in a bipartite graph where each vertex can be matched to at most one in the opposite partition.

🧠 Method (Using Flow Network):

1. Construct a flow network:
   - Add a super source `s` and a super sink `t`
   - Connect `s` to all nodes in left partition (capacity = 1)
   - Connect all nodes in right partition to `t` (capacity = 1)
   - Keep edge capacities between left ↔ right = 1
2. Run Ford-Fulkerson (or any max-flow algorithm)
3. The value of max flow = size of max matching

🧪 Example:

Left (L): A, B
Right (R): 1, 2
Edges: A–1, A–2, B–2

Flow network built:

```
css

s   A, B
A   1, 2
B   2
1, 2   t
```

Run Ford-Fulkerson:

- Matching: A–1, B–2
  ✅ Max Matching = 2

🚰 Ford-Fulkerson Method – Problems

🔹 Problem 1: Max Flow – Basic Network

You are given the following directed graph with capacities:

```
css

s → A (10), s → B (5)
A → B (15), A → t (10)
B → t (10)
```

Task:
Use Ford-Fulkerson Method to compute the maximum flow from `s` to `t`.

- Show all augmenting paths
- Final value of max flow

🔹 Problem 2: Residual Graph Construction

Given the following flow on a network:

```
less

s → A (capacity=10, flow=7)
A → t (capacity=10, flow=7)
```

Task:

- Construct the residual graph
- Identify any augmenting path from `s` to `t`
- What is the remaining possible flow?

🔄 Bipartite Maximum Matching – Problems

🔹 Problem 3: Matching from Flow

Bipartite Graph:

Left Partition (L): A, B, C
Right Partition (R): 1, 2, 3
Edges: A–1, A–2, B–2, C–2, C–3

Task:

- Construct the equivalent flow network
- Use max-flow logic to determine the maximum matching

🔹 Problem 4: Max Matching Condition

Given a bipartite graph where:

- L = {X, Y}
- R = {1, 2}
- Edges: X–1, X–2, Y–1, Y–2

Task:

- What is the maximum matching size?
- Can you find two different matching sets of maximum size?

DSAT-80%

📌 Longest Common Subsequence (LCS)

🔍 Definition:

Given two sequences (strings or arrays), find the length of their longest subsequence that appears in the same order in both (but not necessarily contiguous).

🧠 Subsequence ≠ Substring
Subsequence allows skipping characters, but order must be preserved.

💡 Example:

```
text

X = "ABCBDAB"
Y = "BDCAB"

LCS = "BCAB" or "BDAB" (both length = 4)
```

⚙️ LCS Algorithm Using Dynamic Programming

Let `X[1...m]` and `Y[1...n]` be the two strings.

Step-by-step:

1. Create a table `dp[0...m][0...n]`, where `dp[i][j]` is LCS of `X[1...i]` and `Y[1...j]`
2. Recurrence relation:

```
text

If X[i] == Y[j]:
    dp[i][j] = dp[i-1][j-1] + 1
Else:
    dp[i][j] = max(dp[i-1][j], dp[i][j-1])
```

3. Initialize `dp[0][*]` and `dp[*][0]` to 0
4. The answer is `dp[m][n]`

🧪 Example:

X = "AGGTAB"
Y = "GXTXAYB"

✅ LCS = "GTAB", length = 4

🔄 Time and Space Complexity

- Time: O(m × n)
- Space: O(m × n) (can be optimized to O(n))

🧩 Practice Problem:

Problem:

X = "ABCDEF"
Y = "AEBDF"

1. Compute the LCS length
2. Recover the actual LCS sequence

🧩 Problem

Given:

- X = `"ABCDEF"`
- Y = `"AEBDF"`

We'll:

1. Build the DP table
2. Get the length of LCS
3. Reconstruct the actual sequence

🧮 Step 1: Create DP Table

Let `m = 6`, `n = 5`
We create a table `dp[0...6][0...5]`

Initialize `dp[0][*] = 0` and `dp[*][0] = 0`

Now fill using the rule:

```
text
```

```
If X[i-1] == Y[j-1]:
    dp[i][j] = dp[i-1][j-1] + 1
Else:
    dp[i][j] = max(dp[i-1][j], dp[i][j-1])
```

Let's fill it:

		A	E	B	D	F
	0	0	0	0	0	0
A	0	1	1	1	1	1
B	0	1	1	2	2	2
C	0	1	1	2	2	2
D	0	1	1	2	3	3
E	0	1	2	2	3	3
F	0	1	2	2	3	4

✅ Step 2: LCS Length

`dp[6][5] = 4`

➡️ Length of LCS = 4

🔁 Step 3: Recover the LCS

Start from `dp[6][5]` and trace back:

```
ini

X = ABCDEF
Y = AEBDF
```

Trace path:

- dp[6][5] (F == F) → include "F"
- dp[5][4] (E ≠ D) → go up
- dp[4][4] (D == D) → include "D"
- dp[3][3] (C ≠ B) → go up
- dp[2][3] (B == B) → include "B"
- dp[1][1] (A == A) → include "A"

✅ Reversed: A, B, D, F
➡️ LCS = "ABDF"

🎉 Final Answer:

- ✅ Length: 4
- ✅ LCS: `"ABDF"`

🧠 1. Class P (Polynomial Time)

- Problems that can be solved efficiently by a deterministic algorithm (normal computer).
- "Efficiently" means the algorithm runs in polynomial time:
  Example: $O(n)$, $O(n^2)$, $O(n^3)$, etc.

✅ Examples:

- Sorting (Merge Sort, QuickSort)
- Shortest Path (Dijkstra's Algorithm)
- Matrix Multiplication
- Graph Traversals (BFS, DFS)

🤯 2. Class NP (Non-deterministic Polynomial time)

- Problems for which a proposed solution can be verified in polynomial time.
- Doesn't mean we know how to solve it fast, but we can check a solution fast.

💡 Think of NP as:
"If someone gives you the answer, you can quickly verify it."

✅ Examples:

- Sudoku solution checking
- Subset sum (Is there a subset that sums to K?)
- Hamiltonian Path (Does one exist?)

💥 3. NP-Complete Problems

These are the hardest problems in NP. A problem `X` is NP-Complete if:

1. `X` is in NP

2. Every problem in NP can be reduced to X in polynomial time
   (means if you can solve X efficiently, you can solve all NP problems efficiently)

💡 NP-Complete problems are like the gatekeepers:
Solve one efficiently → you break all of NP wide open.

✅ Famous NP-Complete Problems:

- Traveling Salesperson Problem (TSP)
- 3-SAT
- Subset Sum
- Vertex Cover
- Knapsack Problem (0/1 variant)

🔄 P vs NP – The Big Question

The biggest unsolved question in computer science:

Does P = NP?

- If yes → All NP problems can be solved fast!
- If no → Some problems will always be hard to solve (even if easy to verify)

👉 We don't know the answer yet.

🔍 Visual Summary:

```java
P   NP

NP-Complete   NP

P = NP? — Unknown
```

🧩 Problem 1: Is This Problem in NP?

Problem:

You are given a list of 100 integers. Is there a subset whose sum is exactly 0?

Task:

- Is this problem in NP?
- If yes, explain why.

✅ Hint to Think About:

- You're not being asked to find the subset, just to check if one exists.
- Imagine someone gives you a subset. Can you verify in polynomial time that its sum is 0?

🧩 Problem 2: NP-Complete Reduction

Problem:

You're designing a puzzle game. In one level, you have a grid of switches, and you want to know if you can assign each switch ON/OFF such that at least one switch is ON in every row.

You're told this problem can be reduced to the 3-SAT problem.

Task:

- What does it mean when we say:
  “This problem can be reduced to 3-SAT in polynomial time”?
- Does that make this problem NP-Complete?

✅ Hint to Think About:

- Reducing a known NP-complete problem (like 3-SAT) to your problem shows your problem is at least as hard as 3-SAT.
- But to claim NP-completeness, you also need to show your problem is in NP.

✅ Problem 1: Is This Problem in NP?

🔁 **Problem Recap:**

Given 100 integers, does a subset exist whose sum is exactly 0?

✅ Answer: Yes, the problem is in NP.

🔍 Reasoning:

To be in NP, we need to verify a proposed solution in polynomial time.

- Suppose someone gives you a subset of the 100 integers.
- You can compute the sum of that subset in O(n) time.
- Then check if the sum equals 0 — also in constant time.

➡️ Since verification is fast (polynomial time), this problem is in NP.

Bonus:

This is known as the Subset Sum Problem, which is also NP-Complete (when generalized).

✅ Problem 2: Reduction to 3-SAT

🔁 Problem Recap:

You have a switch-grid puzzle. You're told the problem can be reduced to 3-SAT in polynomial time.

✅ What does it mean?

If any instance of 3-SAT can be transformed into an instance of your puzzle problem in polynomial time...

...then your problem is at least as hard as 3-SAT.

✅ Does that make your problem NP-Complete?

Not automatically. You still need to prove:

1. The problem is in NP (verifiable in polynomial time) ✅
2. A known NP-Complete problem (e.g., 3-SAT) reduces to your problem ✅

If both conditions are true → ✅ Your problem is NP-Complete

💡 Conclusion:

- A reduction from 3-SAT shows hardness.
- You need to also show it's in NP to complete the proof of NP-Completeness.

DSAT-80%

🧩 Problem: Classify the Problem – NP, NP-Complete, or NP-Hard?

You are given the following decision problem:

"Given a list of cities and distances between each pair, is there a tour that visits each city exactly once and returns to the starting city, with total distance ⩽ K?"

(This is the decision version of the Traveling Salesperson Problem, TSP.)

❓ Task:

Classify this problem as one of the following:

- A. P
- B. NP
- C. NP-Complete
- D. NP-Hard

And explain why.